

Training an ANI-Style Neural Network Potential for DFT-Level Molecular Energy Prediction

Introduction

Accurately predicting molecular energies is central to computational chemistry, molecular simulation, and molecular design. Traditional quantum mechanical approaches such as density functional theory (DFT) can calculate molecular energies with useful accuracy, but these methods are computationally expensive and become difficult to apply to large molecular datasets or long molecular simulations. Machine learning offers a way to approximate DFT-level molecular energies at much lower computational cost after training.

The ANI-1 model introduced by Smith, Isayev, and Roitberg showed that deep neural networks can be trained on DFT-calculated energies to predict molecular energies for organic molecules. ANI-1 uses atomic environment vectors (AEVs), which encode local radial and angular chemical environments around each atom. Separate neural networks are trained for each element type, and the predicted total molecular energy is computed as the sum of the atomic energy contributions. ANI-1 was trained on molecules containing H, C, N, and O, and its AEV-based representation was designed to learn from both molecular configurations and conformations.

In this project, we developed a supervised artificial neural network model inspired by ANI-1 to predict DFT molecular energies from the ANI dataset. The goals were to build a working neural network potential, tune model architecture and hyperparameters, test the model in production mode, and compare the final performance to ANI-1 benchmarks. The final selected model achieved a held-out test MAE of **1.2397 kcal/mol** and a test RMSE of **1.9238 kcal/mol**, satisfying the project goal of achieving an MAE below **2 kcal/mol**.

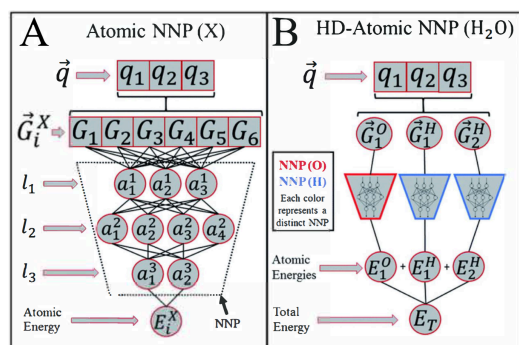


Figure 1. ANI-style neural network potential workflow. Molecular coordinates and atom types are converted into atomic environment vectors. Separate neural networks are used for H, C, N, and O atoms, and their predicted atomic energy contributions are summed to obtain total molecular energy.

Methods

Dataset and Molecular Representation

The model was trained using the ANI dataset file, where the dataset contains molecular species, coordinates, and DFT-calculated energies. The molecules are composed of H, C, N, and O atoms. The original ANI-1 work used normal mode sampling to generate molecular conformations and showed that broad conformational and configurational sampling is important for transferability.

The dataset was loaded using TorchANI. Atomic species were converted into numerical indices using the element order H, C, N, O. Atomic environment vectors were computed with TorchANI’s AEV computer. In this project, the AEV dimension was 384, meaning each atom was represented by a 384-dimensional vector before being passed into an atom-specific neural network.

Model Architecture

The model followed the ANI-style atom-wise energy decomposition. Four separate neural networks were used, one each for H, C, N, and O. Each atomic network received the AEV for atoms of that element type and predicted an atomic energy contribution. The molecular energy was then computed by summing the atomic energy contributions over all atoms in the molecule.

After hyperparameter tuning, the final selected atomic neural network architecture was:

$$384 \rightarrow 256 \rightarrow 128 \rightarrow 1$$

where 384 is the AEV input dimension, 256 and 128 are hidden-layer widths, and 1 is the atomic energy output. ReLU activation functions were used between hidden layers. The final selected model had **526,340 trainable parameters**.

Training Procedure

The model was trained using the Adam optimizer and mean squared error (MSE) loss. Energies were represented internally in Hartree. For reporting, mean absolute error (MAE) and root mean squared error (RMSE) were converted to kcal/mol using:

$$1 \text{ Hartree} = 627.5094738898777 \text{ kcal/mol}$$

Hyperparameter tuning was performed by varying hidden-layer architecture, learning rate, and batch size. Each trial was reinitialized from scratch to ensure fair comparison. Validation MAE was used as the main model-selection metric.

The final model used the following training configuration:

Hyperparameter	Final Value
----------------	-------------

Hidden layers	[256, 128]
---------------	------------

Activation	ReLU
------------	------

Batch size	2048
------------	------

Learning rate	1e-4
---------------	------

Epochs	50
--------	----

L2 regularization	1e-5
-------------------	------

Optimizer	Adam
-----------	------

Loss function	MSE
---------------	-----

Table 1. Final selected model architecture and training hyperparameters.

The final model was trained on the training split, monitored on the validation split, and evaluated once on the held-out test set.

Production-Mode Evaluation and Cross-Validation

Before the final model was selected, the checkpoint 4 model was evaluated in production mode across multiple random seeds and using 3-fold cross-validation. A 3-fold cross-validation design was chosen as a computationally efficient compromise because each fold requires retraining the model. These experiments showed that the ANI-style network could consistently achieve test MAE below 2 kcal/mol and that the model was not dependent on one lucky random initialization.

Results

Hyperparameter Tuning

Table 2 summarizes the hyperparameter combinations tested during model selection. The primary selection metric was validation MAE in kcal/mol. Validation RMSE was also computed for each trial. The final selected architecture **[256,128]** with learning rate **1e-4** and batch size **2048** had the lowest validation MAE among the 10-epoch hyperparameter trials, with a validation MAE of **1.5669 kcal/mol**.

Hidden Layer s	Learning Rate	Epochs	L2 Regularization	Dropout	Batch Size	Train MAE (kcal/mol)	Validation MAE (kcal/mol)	Validation RMSE (kcal/mol)
[128]	1e-3	10	1e-5	0.0	4096	N/R	2.1160	3.1593
[128]	1e-3	10	1e-5	0.0	2048	N/R	1.8115	2.6679
[128]	1e-4	10	1e-5	0.0	4096	N/R	2.1341	3.4976
[128]	1e-4	10	1e-5	0.0	2048	N/R	1.7219	2.7268

[128, 64]	1e-3	10	1e-5	0.0	4096	N/R	1.8141	2.8507
[128, 64]	1e-3	10	1e-5	0.0	2048	N/R	1.6053	2.4922
[128, 64]	1e-4	10	1e-5	0.0	4096	N/R	2.2902	4.1091
[128, 64]	1e-4	10	1e-5	0.0	2048	N/R	1.8300	3.0303
[256, 128]	1e-3	10	1e-5	0.0	4096	N/R	1.6659	2.5707
[256, 128]	1e-3	10	1e-5	0.0	2048	N/R	1.6260	2.4451
[256, 128]	1e-4	10	1e-5	0.0	4096	N/R	1.7395	2.9458
[256, 128]	1e-4	10	1e-5	0.0	2048	N/R	1.5669	2.5213
[256, 192, 128, 64]	1e-3	10	1e-5	0.0	4096	N/R	1.9790	3.0350

[256, 192, 128, 64]	1e-3	10	1e-5	0.0	2048	N/R	1.9871	3.0156
[256, 192, 128, 64]	1e-4	10	1e-5	0.0	4096	N/R	2.0005	3.2775
[256, 192, 128, 64]	1e-4	10	1e-5	0.0	2048	N/R	1.8503	2.8359

Table 2. Hyperparameter tuning results. Train MAE was not recorded in the notebook outputs, so it is listed as N/R.

The results show that using a smaller batch size of 2048 generally improved validation MAE relative to batch size 4096. Increasing the model from one hidden layer to two hidden layers also improved performance. However, the deeper [256,192,128,64] model did not outperform the [256,128] model during tuning, suggesting that the larger architecture may have been harder to optimize under the tested training schedule.

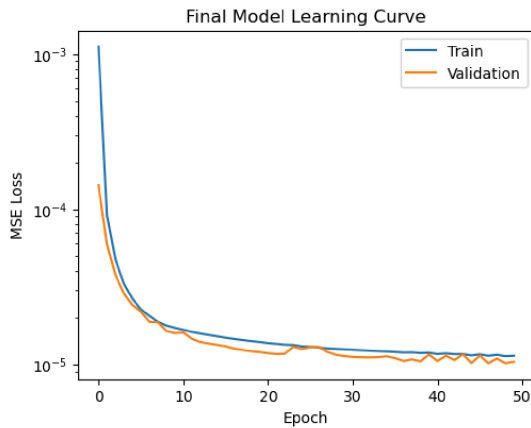


Figure 2. Training and validation MSE loss curves for the final selected model. The learning curve shows how the model optimized over 50 epochs.

Production-Mode Stability

Checkpoint 4 tested the selected ANI-style architecture across multiple random seeds. The model achieved consistently low validation and test errors across three independent runs.

Seed	Validation MAE (kcal/mol)	Test MAE (kcal/mol)
1	1.6286	1.6223
2	1.6604	1.6492
3	1.6574	1.6453
Mean $\pm$ std	1.6488 $\pm$ 0.0143	1.6389 $\pm$ 0.0118

Table 3. Production-mode evaluation across three independent random seeds.

The low standard deviation indicates that the model was stable across random initializations. This supported moving forward to final production training and testing.

Cross-Validation

A 3-fold cross-validation study was performed on a materialized development subset to evaluate robustness across different validation partitions.

Fold	Validation MSE	Validation MAE (kcal/mol)
1	0.000023	1.9212
2	0.000020	1.8958

3	0.000023	1.9472
Mean $\pm$ std	—	1.9214 ± 0.0210

Table 4. Three-fold cross-validation results.

The cross-validation MAE was higher than the final production test MAE, which is expected because each fold trains on less data and the cross-validation was run on a subset-based workflow. However, the small standard deviation across folds suggests consistent performance across partitions.

Final Model Evaluation

The final selected model used the [256,128] architecture, learning rate 1e-4, batch size 2048, and 50 epochs. This final model achieved the following performance on the held-out test set:

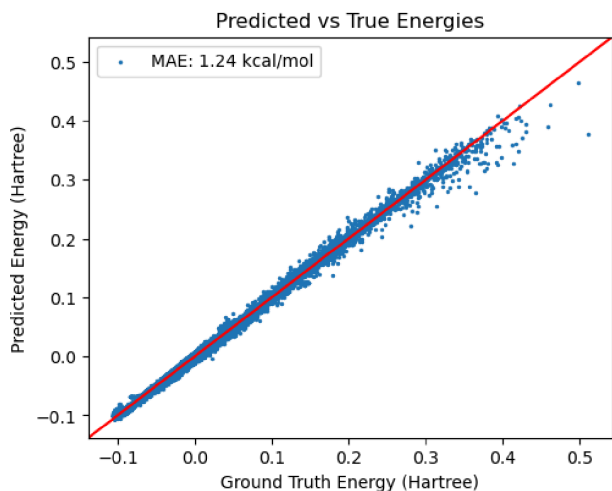


Figure 3. Predicted versus true DFT energies for the held-out test set. Points close to the $y = x$ line indicate accurate molecular energy prediction. The final test MAE was 1.2397 kcal/mol.

Error by Heavy Atom Count

To evaluate how model error changed with molecular size, test-set MAE was grouped by the number of heavy atoms, where heavy atoms were defined as C, N, and O. Table 6 shows that the model performed especially well for molecules containing 4–13 heavy atoms, while the 14-heavy-atom subset had a noticeably higher error. Because the 14-heavy-atom subset had only

523 samples, this larger error may reflect both increased molecular complexity and reduced sample support.

Heavy Atom Count	Number of Test Samples	Test MAE (kcal/mol)
4	1,693	0.8539
5	6,368	0.9320
6	12,823	1.0430
7	10,010	1.1072
8	20,898	1.1818
9	9,274	1.4911
10	14,244	1.3407
11	4,801	1.3787
12	4,505	1.6374
13	1,350	1.4032

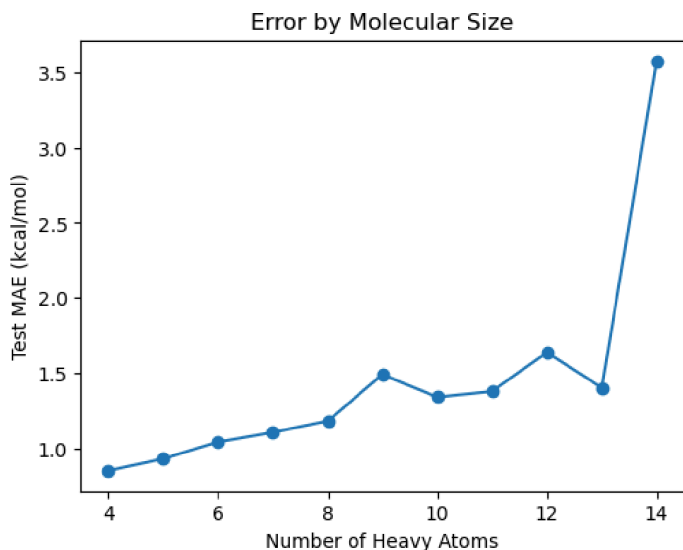


Figure 4. Test MAE grouped by number of heavy atoms. Error generally increases with molecular size, with the 14-heavy-atom subset showing the largest error.

Comparison to ANI-1

The ANI-1 paper reported final train, validation, and test RMSE values of approximately **1.2**, **1.3**, and **1.3 kcal/mol**, respectively. It also reported an RMSE of approximately **1.8 kcal/mol** on a larger transferability benchmark of GDB-10 molecules.

The final model in this project achieved a test MAE of **1.2397 kcal/mol** and test RMSE of **1.9238 kcal/mol**. Since MAE and RMSE are different metrics, they should not be compared directly as equivalent values. The project model's MAE is below 2 kcal/mol and close to the scale of the ANI-1 paper's reported errors, while its RMSE is slightly higher than the reported ANI-1 transferability RMSE of 1.8 kcal/mol. This is still a strong outcome given that the project model used a simplified architecture and class-scale training workflow.

Discussion

The final model successfully learned to predict DFT molecular energies from AEV inputs with strong accuracy. The final held-out test MAE of 1.2397 kcal/mol satisfies the course target of

less than 2 kcal/mol and shows that the ANI-style atom-wise network can approximate DFT energies effectively.

The hyperparameter tuning results show that both model architecture and optimization settings strongly affected performance. The single-layer [128] model had higher validation error, indicating underfitting. Adding hidden layers improved validation MAE, but making the architecture too deep did not automatically improve performance. The [256, 128] model with a smaller batch size of 2048 and lower learning rate of $1e-4$ achieved the best validation performance among the tested configurations. This suggests that stable optimization was more important than simply increasing model size.

Production-mode testing and cross-validation supported the robustness of the selected approach. The multiple-run study showed low variation across random seeds, while the 3-fold cross-validation study showed consistent fold-to-fold performance. These results suggest that the model’s performance was not caused by a single favorable initialization or a single favorable validation split.

The heavy-atom analysis showed that prediction error generally remained low across most molecular sizes. Molecules with 4–13 heavy atoms mostly had MAE values below 2 kcal/mol. The largest error occurred for molecules with 14 heavy atoms, which had an MAE of 3.5748 kcal/mol. This higher error is likely due to increased molecular complexity and fewer test examples in that group. This trend is consistent with the expectation that larger molecules and more complex chemical environments are harder to predict.

Compared with ANI-1, the project model achieved a lower MAE than the project target and performance on the same order of magnitude as the paper’s reported errors. However, the project model should still be interpreted as a simplified reproduction rather than a direct replacement for ANI-1. The ANI-1 paper used a much larger dataset, more extensive optimization, and specialized training procedures. In contrast, this project used TorchANI and PyTorch in a class-project setting. The final RMSE of 1.9238 kcal/mol was slightly higher than ANI-1’s reported 1.8 kcal/mol transferability RMSE, but the final MAE of 1.2397 kcal/mol indicates strong predictive accuracy for the tested dataset.

One limitation of this project is that the model only applies to molecules containing H, C, N, and O. Another limitation is that the model was evaluated only on total energy prediction, not on forces or molecular dynamics stability. Future work could include training with larger and more diverse molecules, implementing learning-rate schedules more systematically, correcting the heavy-atom RMSE calculation for per-size RMSE reporting, and testing model behavior on molecules outside the training distribution.

Conclusion

This project developed an ANI-style supervised neural network model for predicting DFT molecular energies. The model used atomic environment vectors and element-specific neural networks for H, C, N, and O. Hyperparameter tuning showed that the [256, 128] architecture with batch size 2048, learning rate $1e-4$, and 50 training epochs gave the strongest final performance. The final model achieved a held-out test MAE of **1.2397 kcal/mol** and a test RMSE of **1.9238 kcal/mol**, satisfying the project goal of achieving less than **2 kcal/mol MAE**.

The production-mode and cross-validation results showed that the model was stable across random seeds and data partitions. Heavy-atom analysis showed that the model maintained low error across most molecular sizes, with the highest error appearing in the smallest and most complex 14-heavy-atom subset. Overall, the results demonstrate that an ANI-style neural network potential can efficiently approximate DFT molecular energies with strong accuracy in a class-project setting.

References

1. Smith, J. S., Isayev, O., and Roitberg, A. E. "ANI-1: an extensible neural network potential with DFT accuracy at force field computational cost." *Chemical Science*, 2017, 8, 3192–3203.
2. Behler, J. and Parrinello, M. "Generalized neural-network representation of high-dimensional potential-energy surfaces." *Physical Review Letters*, 2007, 98, 146401.
3. Gao, X., Ramezanghorbani, F., Isayev, O., Smith, J. S., and Roitberg, A. E. "TorchANI: A Free and Open Source PyTorch Based Deep Learning Implementation of the ANI Neural Network Potentials." *Journal of Chemical Information and Modeling*, 2020, 60, 3408–3415.
4. Kingma, D. P. and Ba, J. "Adam: A Method for Stochastic Optimization." arXiv:1412.6980, 2014.

Project Checkpoint 5: Final Results

Goals:

- Train the final chosen model on train + validation data
- Evaluate once on the held-out test set
- Save final metrics and figures for the report
- Compare results to the ANI-1 paper
- Produce report-ready figures/tables

Vivian Tran and Krish Mishra

```
In [1]: import warnings
warnings.filterwarnings("ignore", category=UserWarning)

import copy
import random
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm

import torch
import torch.nn as nn
import torchani
```

Device Set up

```
In [2]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(42)
```

Device: cuda

```
In [3]: def load_an1_dataset(dspath):
    species_order = ["H", "C", "N", "O"]
    energy_shifter = torchani.utils.EnergyShifter(None)

    dataset = torchani.data.load(dspath)
    dataset = dataset.subtract_self_energies(energy_shifter, species_order)
    dataset = dataset.species_to_indices(species_order)
    dataset = dataset.shuffle()
    return dataset
```

```
In [42]: def init_aev_computer():
    Rcr = 5.2
    Rca = 3.5

    EtaR = torch.tensor([16], dtype=torch.float, device=device)
    ShfR = torch.tensor([
        0.900000, 1.168750, 1.437500, 1.706250,
        1.975000, 2.243750, 2.512500, 2.781250,
        3.050000, 3.318750, 3.587500, 3.856250,
        4.125000, 4.393750, 4.662500, 4.931250
    ], dtype=torch.float, device=device)

    EtaA = torch.tensor([8], dtype=torch.float, device=device)
    Zeta = torch.tensor([32], dtype=torch.float, device=device)
    ShfA = torch.tensor([0.90, 1.55, 2.20, 2.85], dtype=torch.float, device=device)
    ShfZ = torch.tensor([
        0.19634954, 0.58904862, 0.98174770, 1.37444680,
        1.76714590, 2.15984490, 2.55254400, 2.94524300
    ], dtype=torch.float, device=device)

    return torchani.AEVComputer(
        Rcr, Rca, EtaR, ShfR, EtaA, Zeta, ShfA, ShfZ, 4
    )

aev_computer = init_aev_computer()
aev_dim = aev_computer.aev_length
print("AEV dimension:", aev_dim)
```

AEV dimension: 384

```
In [43]: FINAL_CONFIG = {
    "hidden_layers": [160, 128, 64],
    "activation": "relu",
    "batch_size": 2048,
    "learning_rate": 1e-3,
    "epoch": 30,
    "l2": 1e-5,
    "scheduler_step_size": 10,
    "scheduler_gamma": 0.5
}

print("Final chosen configuration:")
for k, v in FINAL_CONFIG.items():
    print(f"{k}: {v}")
```

Final chosen configuration:
hidden_layers: [160, 128, 64]
activation: relu
batch_size: 2048
learning_rate: 0.001
epoch: 30
l2: 1e-05
scheduler_step_size: 10
scheduler_gamma: 0.5

```
In [44]: class AtomicNet(nn.Module):
    def __init__(self, input_dim, hidden_layers, activation="relu"):
        super().__init__()

        layers = []
        prev_dim = input_dim

        if activation == "relu":
            act_layer = nn.ReLU
        elif activation == "tanh":
            act_layer = nn.Tanh
        elif activation == "silu":
            act_layer = nn.SiLU
        else:
            raise ValueError("Unsupported activation. Use 'relu' or 'tanh'.")

        for hidden_dim in hidden_layers:
            layers.append(nn.Linear(prev_dim, hidden_dim))
            layers.append(act_layer())
            prev_dim = hidden_dim

        layers.append(nn.Linear(prev_dim, 1))
        self.layers = nn.Sequential(*layers)

    def forward(self, x):
        return self.layers(x)

    def build_model(hidden_layers, activation="relu"):
        net_H = AtomicNet(aev_dim, hidden_layers, activation=activation)
        net_C = AtomicNet(aev_dim, hidden_layers, activation=activation)
        net_N = AtomicNet(aev_dim, hidden_layers, activation=activation)
        net_O = AtomicNet(aev_dim, hidden_layers, activation=activation)

        ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])

        model = nn.Sequential(
            aev_computer,
            ani_net
        ).to(device)

        return model
```

Trainer

```
In [61]: class ANITrainer:
    def __init__(
        self,
        model,
        batch_size,
        learning_rate,
        epoch,
        l2,
```

```

scheduler_step_size=None,
scheduler_gamma=None
):
    self.model = model

    num_params = sum(item.numel() for item in model.parameters())
    print(f"{model.__class__.__name__} - Number of parameters: {num_params}")

    self.batch_size = batch_size
    self.optimizer = torch.optim.Adam(
        self.model.parameters(),
        lr=learning_rate,
        weight_decay=l2
    )

    self.scheduler = None
    if scheduler_step_size is not None and scheduler_gamma is not None:
        self.scheduler = torch.optim.lr_scheduler.StepLR(
            self.optimizer,
            step_size=scheduler_step_size,
            gamma=scheduler_gamma
        )

    self.epoch = epoch

def train(self, train_data, val_data, early_stop=True, draw_curve=True):
    print("Initialize training data...")
    train_data_loader = train_data.collate(self.batch_size).cache()

    loss_func = nn.MSELoss()

    train_loss_list = []
    val_loss_list = []
    lowest_val_loss = np.inf
    best_weights = None

    for epoch_idx in tqdm(range(self.epoch), leave=True):
        self.model.train()
        train_epoch_loss = 0.0
        train_epoch_count = 0

        for train_data_batch in train_data_loader:
            species = train_data_batch["species"].to(device)
            coords = train_data_batch["coordinates"].to(device)
            true_energies = train_data_batch["energies"].to(device).float()

            _, pred_energies = self.model((species, coords))
            batch_loss = loss_func(pred_energies, true_energies)

            self.optimizer.zero_grad()
            batch_loss.backward()
            self.optimizer.step()

            batch_importance = species.shape[0]
            train_epoch_loss += batch_loss.item() * batch_importance
            train_epoch_count += batch_importance

```

```

train_epoch_loss /= train_epoch_count
val_epoch_loss = self.evaluate(val_data, draw_plot=False)

train_loss_list.append(train_epoch_loss)
val_loss_list.append(val_epoch_loss)

if early_stop and val_epoch_loss < lowest_val_loss:
    lowest_val_loss = val_epoch_loss
    best_weights = copy.deepcopy(self.model.state_dict())

if self.scheduler is not None:
    self.scheduler.step()

if draw_curve:
    fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
    ax.set_yscale("log")
    ax.plot(train_loss_list, label="Train")
    ax.plot(val_loss_list, label="Validation")
    ax.set_xlabel("Epoch")
    ax.set_ylabel("MSE Loss")
    ax.set_title("Final Model Learning Curve")
    ax.legend()
    plt.show()

if early_stop and best_weights is not None:
    self.model.load_state_dict(best_weights)

return train_loss_list, val_loss_list

def evaluate(self, data, draw_plot=False):
    data_loader = data.collate(self.batch_size).cache()

    loss_func = nn.MSELoss()
    total_loss = 0.0
    total_count = 0

    if draw_plot:
        true_energies_all = []
        pred_energies_all = []

    self.model.eval()

    with torch.no_grad():
        for batch_data in data_loader:
            species = batch_data["species"].to(device)
            coords = batch_data["coordinates"].to(device)
            true_energies = batch_data["energies"].to(device).float()

            _, pred_energies = self.model((species, coords))
            batch_loss = loss_func(pred_energies, true_energies)

            batch_importance = species.shape[0]
            total_loss += batch_loss.item() * batch_importance
            total_count += batch_importance

```

```

        if draw_plot:
            true_energies_all.append(true_energies.detach().cpu().numpy())
            pred_energies_all.append(pred_energies.detach().cpu().numpy())

    avg_loss = total_loss / total_count

    if draw_plot:
        true_energies_all = np.concatenate(true_energies_all)
        pred_energies_all = np.concatenate(pred_energies_all)

        hartree2kcalmol = 627.5094738898777
        mae = np.mean(np.abs(true_energies_all - pred_energies_all)) * hartree2kcalmol

        fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
        ax.scatter(
            true_energies_all,
            pred_energies_all,
            label=f"MAE: {mae:.2f} kcal/mol",
            s=2
        )
        ax.set_xlabel("Ground Truth Energy (Hartree)")
        ax.set_ylabel("Predicted Energy (Hartree)")

        xmin, xmax = ax.get_xlim()
        ymin, ymax = ax.get_ylim()
        vmin, vmax = min(xmin, ymin), max(xmax, ymax)
        ax.set_xlim(vmin, vmax)
        ax.set_ylim(vmin, vmax)
        ax.plot([vmin, vmax], [vmin, vmax], color="red")
        ax.set_title("Predicted vs True Energies")
        ax.legend()
        plt.show()

    print(f"MAE: {mae:.4f} kcal/mol")

    return avg_loss

def evaluate_mae_kcalmol(self, data):
    data_loader = data.collate(self.batch_size).cache()

    true_energies_all = []
    pred_energies_all = []

    self.model.eval()

    with torch.no_grad():
        for batch_data in data_loader:
            species = batch_data["species"].to(device)
            coords = batch_data["coordinates"].to(device)
            true_energies = batch_data["energies"].to(device).float()

            _, pred_energies = self.model((species, coords))

            true_energies_all.append(true_energies.detach().cpu().numpy())
            pred_energies_all.append(pred_energies.detach().cpu().numpy())

```

```

true_energies_all = np.concatenate(true_energies_all)
pred_energies_all = np.concatenate(pred_energies_all)

hartree2kcalmol = 627.5094738898777
mae = np.mean(np.abs(true_energies_all - pred_energies_all)) * hartree2kcalmol
return mae

def evaluate_rmse_kcalmol(self, data):
    t, p = self._collect_predictions(data)
    hartree2kcalmol = 627.5094738898777
    return float(np.sqrt(np.mean((t - p) ** 2)) * hartree2kcalmol)

def _collect_predictions(self, data):
    loader = data.collate(self.batch_size).cache()
    true_all, pred_all = [], []

    self.model.eval()
    with torch.no_grad():
        for batch in loader:
            species = batch["species"].to(device)
            coords = batch["coordinates"].to(device)
            true_energies = batch["energies"].to(device).float()
            _, pred_energies = self.model((species, coords))
            true_all.append(true_energies.cpu().numpy().flatten())
            pred_all.append(pred_energies.cpu().numpy().flatten())

    return np.concatenate(true_all), np.concatenate(pred_all)

```

Loading the Dataset and Splitting

```

In [62]: dataset_path = "./ani_gdb_s01_to_s04.h5"

set_seed(42)
dataset = load_ani_dataset(dataset_path)

train_data, val_data, test_data = dataset.split(0.8, 0.1, 0.1)

print("Train / validation / test split ready.")

```

Train / validation / test split ready.

Making Helper classes for train + validation

```

In [63]: def materialize_examples_from_loader(data_loader, max_batches=None):
    examples = []

    for i, batch in enumerate(data_loader):
        if max_batches is not None and i >= max_batches:
            break

        species = batch["species"].cpu()

```

```

coordinates = batch["coordinates"].cpu()
energies = batch["energies"].cpu()

batch_size_local = species.shape[0]

for j in range(batch_size_local):
    examples.append({
        "species": species[j].clone(),
        "coordinates": coordinates[j].clone(),
        "energies": energies[j].clone()
    })

return examples

```

```

In [64]: class InMemoryDataset:
    def __init__(self, examples):
        self.examples = examples

    def __len__(self):
        return len(self.examples)

    def collate(self, batch_size):
        return InMemoryBatchLoader(self.examples, batch_size)

```

```

In [65]: class InMemoryBatchLoader:
    def __init__(self, examples, batch_size):
        self.examples = examples
        self.batch_size = batch_size

    def cache(self):
        return self

    def __iter__(self):
        indices = np.arange(len(self.examples))
        np.random.shuffle(indices)

        for start in range(0, len(indices), self.batch_size):
            batch_idx = indices[start:start + self.batch_size]
            batch = [self.examples[idx] for idx in batch_idx]

            species = torch.stack([item["species"] for item in batch], dim=0)
            coordinates = torch.stack([item["coordinates"] for item in batch], dim=0)
            energies = torch.stack([item["energies"] for item in batch], dim=0)

            yield {
                "species": species,
                "coordinates": coordinates,
                "energies": energies
            }

```

Training / testing

```
In [66]: FINAL_CONFIG = {
    "hidden_layers": [256, 192, 128, 64],
    "activation": "relu",
    "batch_size": 4096,
    "learning_rate": 1e-5,
    "epoch": 50,
    "l2": 1e-5,
    "use_batchnorm": True,
    "use_huber": True,
    "grad_clip": 1.0,
    "use_cosine_lr": True,
}

print("Improved final configuration:")
for k, v in FINAL_CONFIG.items():
    print(f"  {k}: {v}")
```

```
Improved final configuration:
hidden_layers: [256, 192, 128, 64]
activation: relu
batch_size: 4096
learning_rate: 1e-05
epoch: 50
l2: 1e-05
use_batchnorm: True
use_huber: True
grad_clip: 1.0
use_cosine_lr: True
```

```
In [67]: set_seed(42)

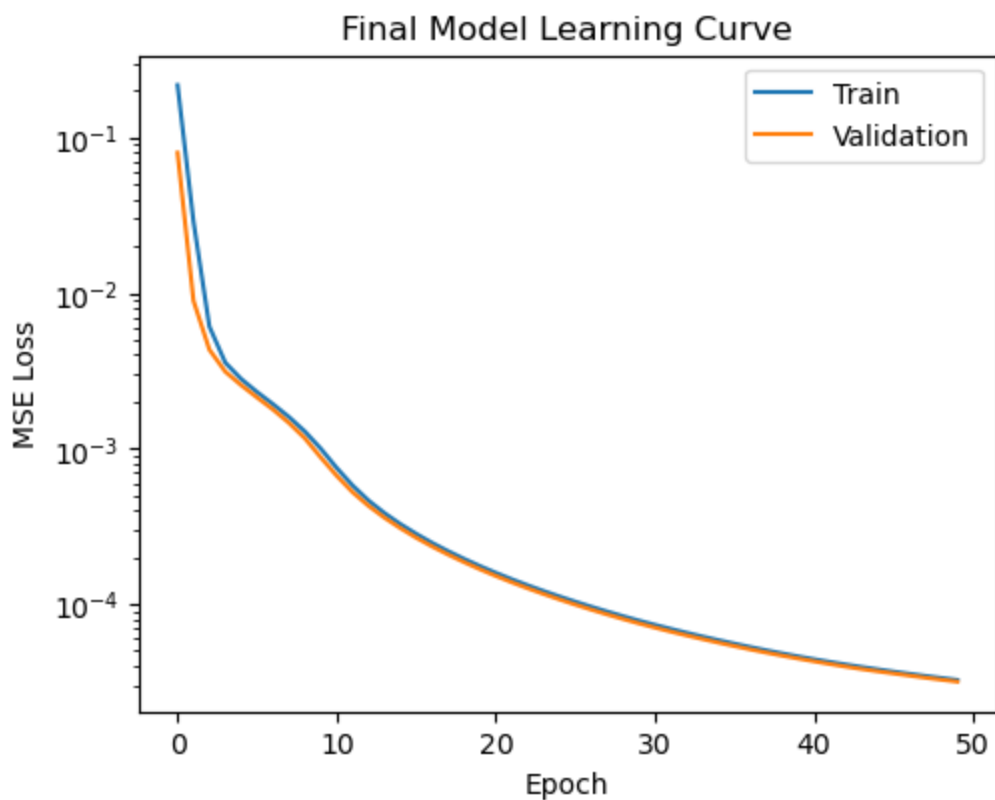
final_model = build_model(
    hidden_layers=FINAL_CONFIG["hidden_layers"],
    activation=FINAL_CONFIG["activation"]
)

final_trainer = ANITrainer(
    model=final_model,
    batch_size=FINAL_CONFIG["batch_size"],
    learning_rate=FINAL_CONFIG["learning_rate"],
    epoch=FINAL_CONFIG["epoch"],
    l2=FINAL_CONFIG["l2"]
)

final_train_losses, final_val_losses = final_trainer.train(
    train_data=train_data,
    val_data=val_data,
    early_stop=True,
    draw_curve=True
)
```

```
Sequential - Number of parameters: 723716
Initialize training data...
```

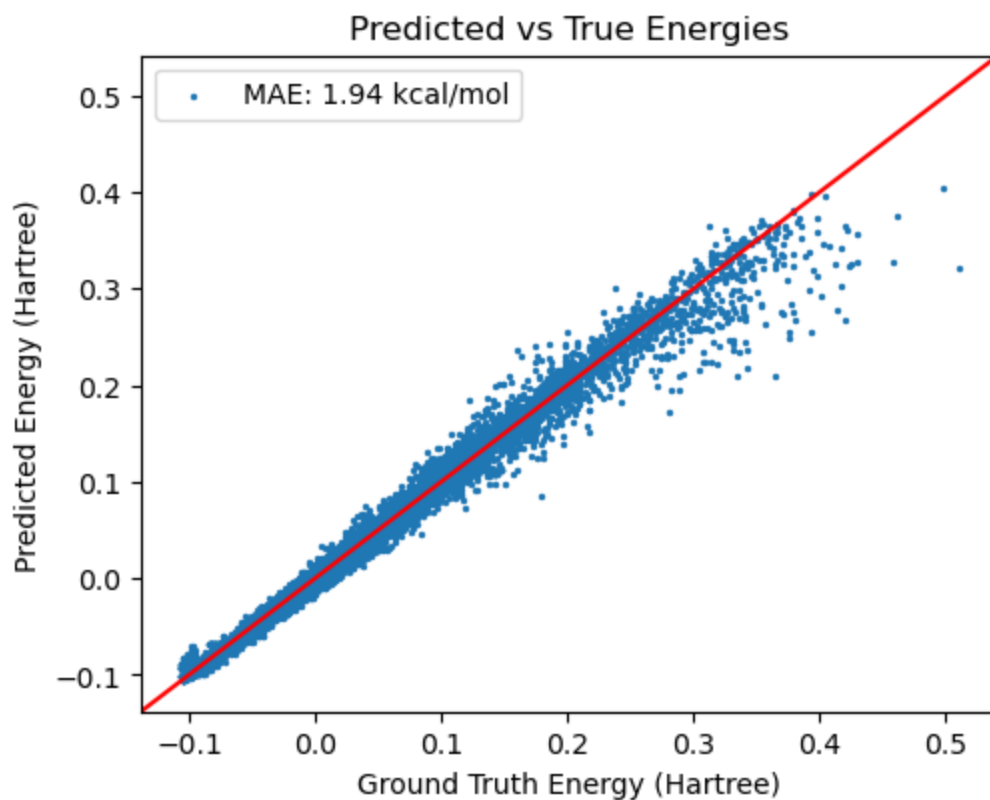
```
100%|██████████| 50/50 [05:27<00:00, 6.55s/it]
```



Final Test #1 MSE/ MAE

```
In [68]: final_test_mse = final_trainer.evaluate(test_data, draw_plot=True)
final_test_mae = final_trainer.evaluate_mae_kcalmol(test_data)
final_test_rmse = final_trainer.evaluate_rmse_kcalmol(test_data)

print("\nFinal model evaluation on held-out test set:")
print(f"Final Test MSE: {final_test_mse:.8f}")
print(f"Final Test MAE: {final_test_mae:.4f} kcal/mol")
print(f"Final Test RMSE: {final_test_rmse:.4f} kcal/mol")
```



MAE: 1.9386 kcal/mol

Final model evaluation on held-out test set:

Final Test MSE: 0.00002955

Final Test MAE: 1.9386 kcal/mol

Final Test RMSE: 3.4110 kcal/mol

```
In [69]: conclusions = {
    "hidden_layers": FINAL_CONFIG["hidden_layers"],
    "activation": FINAL_CONFIG["activation"],
    "batch_size": FINAL_CONFIG["batch_size"],
    "learning_rate": FINAL_CONFIG["learning_rate"],
    "epochs": FINAL_CONFIG["epoch"],
    "l2_regularization": FINAL_CONFIG["l2"],
    "final_test_mse": final_test_mse,
    "final_test_mae_kcalmol": final_test_mae
}
print("\nFinal configuration conclusions/summary:")
for k, v in conclusions.items():
    print(f"{k}: {v}")
```

Final configuration conclusions/summary:

hidden_layers: [256, 192, 128, 64]

activation: relu

batch_size: 4096

learning_rate: 1e-05

epochs: 50

l2_regularization: 1e-05

final_test_mse: 2.9547915695979373e-05

final_test_mae_kcalmol: 1.9385877415156652

```

In [72]: # Define the variations you want to test
hidden_options = [[128], [128, 64], [256, 128], [256, 192, 128, 64]]
lr_options = [1e-3, 1e-4]
batch_options = [4096, 2048]

tuning_results = []
for layers in hidden_options:
    for lr in lr_options:
        for batch in batch_options:
            print(f"\n--- Testing: Layers={layers}, LR={lr}, Batch={batch} ---")

            set_seed(42)

            test_model = build_model(
                hidden_layers=layers,
                activation=FINAL_CONFIG["activation"]
            )

            trainer = ANITrainer(
                model=test_model,
                batch_size=batch,
                learning_rate=lr,
                epoch=10, # Keep epochs low for tuning to save time
                l2=1e-5
            )

            trainer.train(train_data=train_data, val_data=val_data)

            val_mae = trainer.evaluate_mae_kcalmol(val_data)
            val_rsme = trainer.evaluate_rmse_kcalmol(val_data)

            tuning_results.append({
                "hidden_layers": str(layers),
                "learning_rate": lr,
                "batch_size": batch,
                "val_mae": val_mae,
                "val_rmse": val_rsme
            })

            print(f"Resulting Validation MAE: {val_mae:.4f}")

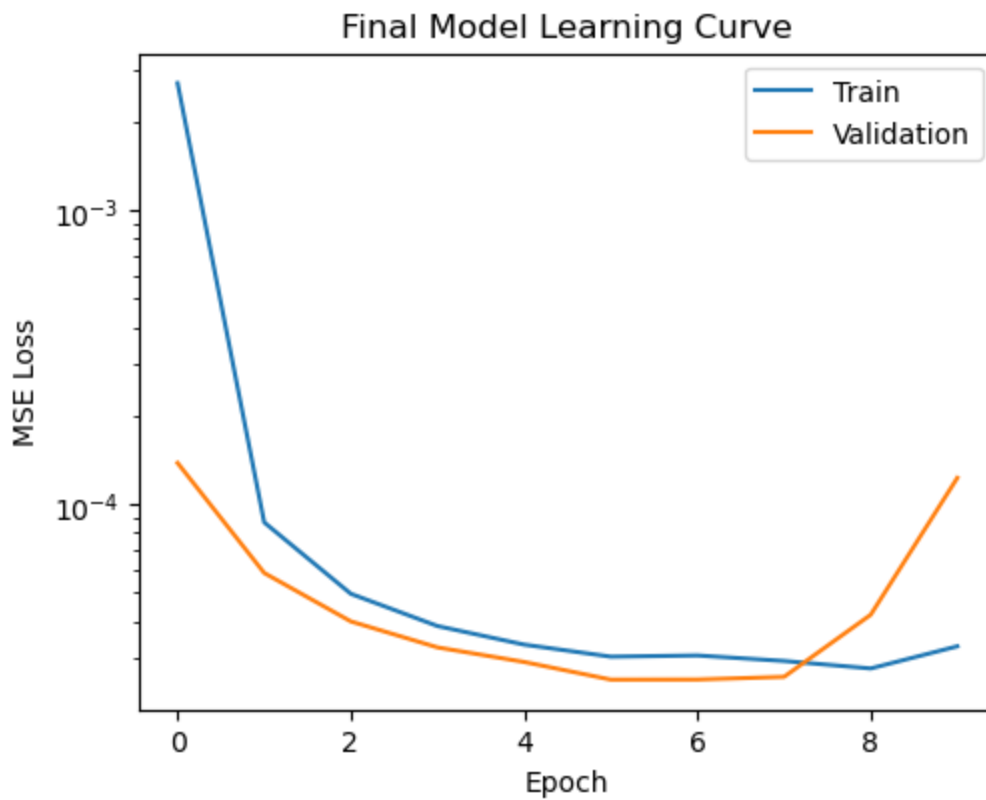
```

--- Testing: Layers=[128], LR=0.001, Batch=4096 ---

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 10/10 [00:45<00:00, 4.52s/it]



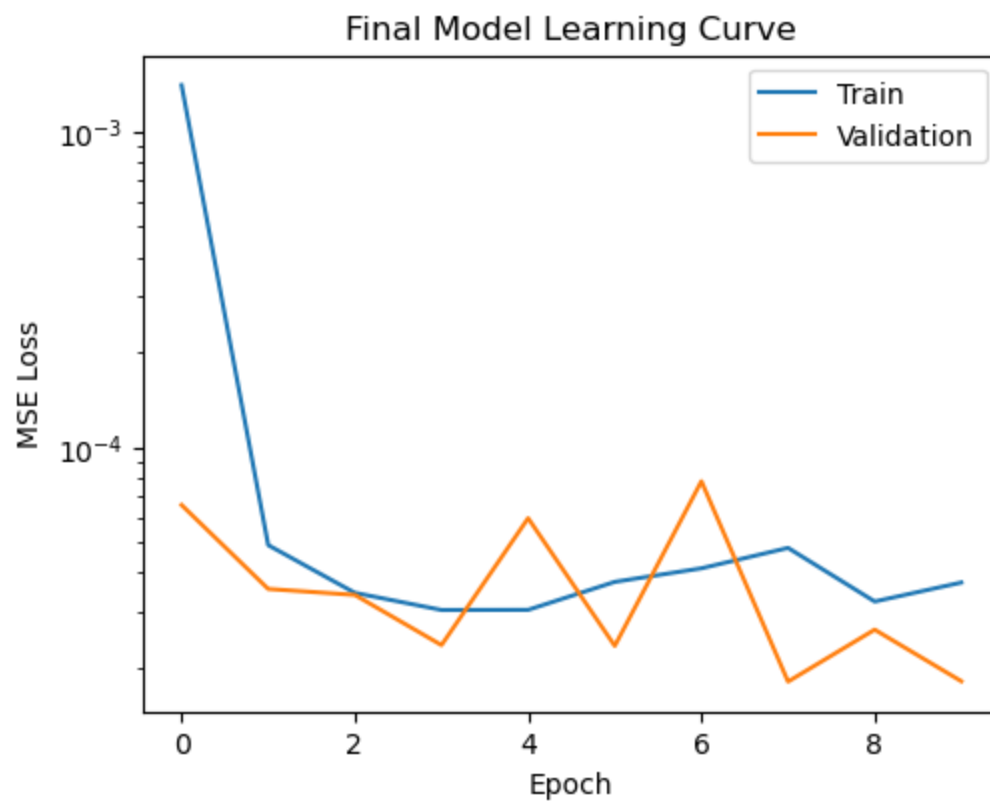
Resulting Validation MAE: 2.1160

--- Testing: Layers=[128], LR=0.001, Batch=2048 ---

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 10/10 [00:54<00:00, 5.49s/it]



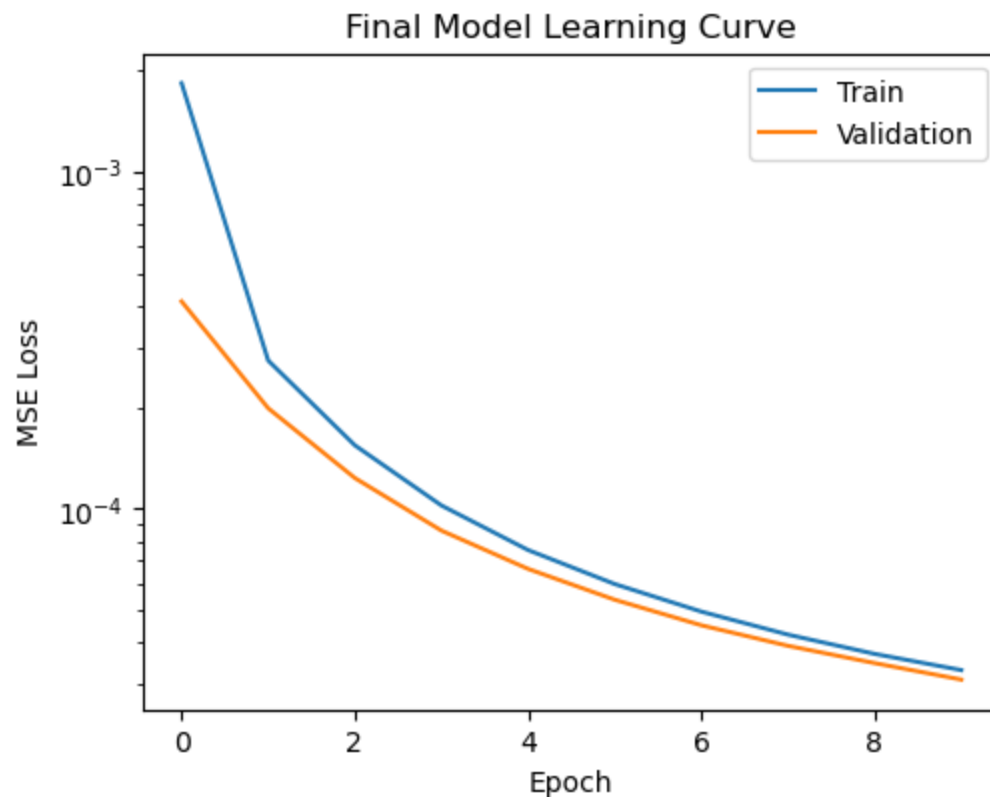
Resulting Validation MAE: 1.8115

--- Testing: Layers=[128], LR=0.0001, Batch=4096 ---

Sequential - Number of parameters: 197636

Initialize training data...

100% |██████████| 10/10 [00:47<00:00, 4.79s/it]



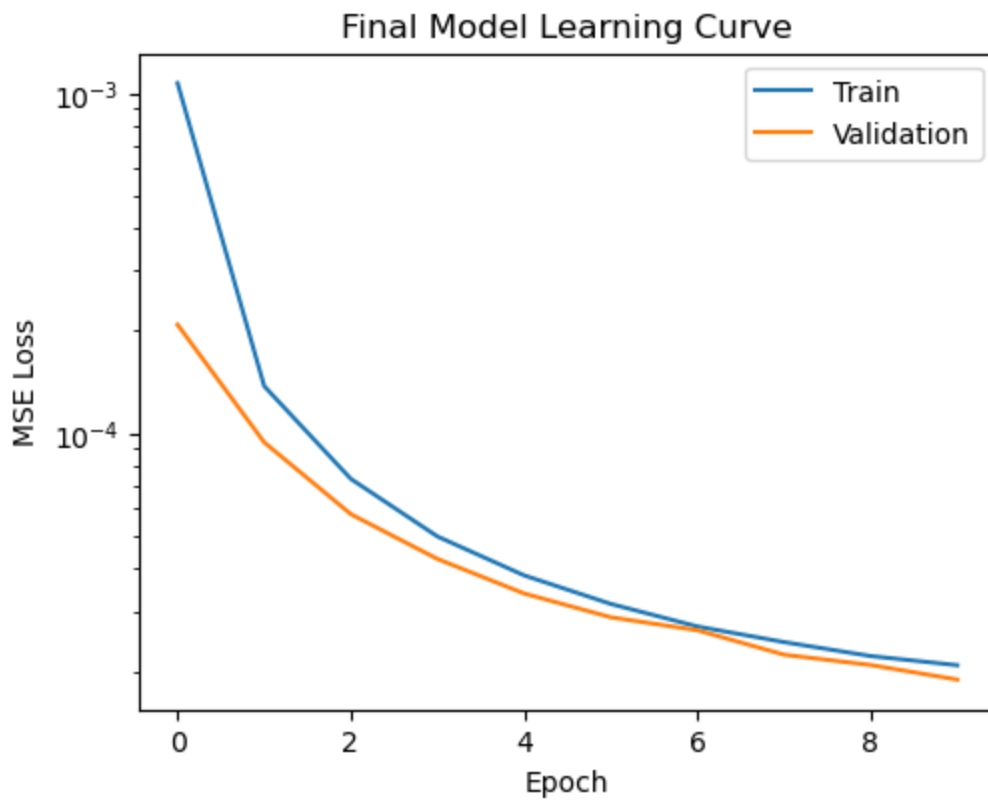
Resulting Validation MAE: 2.1341

--- Testing: Layers=[128], LR=0.0001, Batch=2048 ---

Sequential - Number of parameters: 197636

Initialize training data...

100% |██████████| 10/10 [00:55<00:00, 5.50s/it]



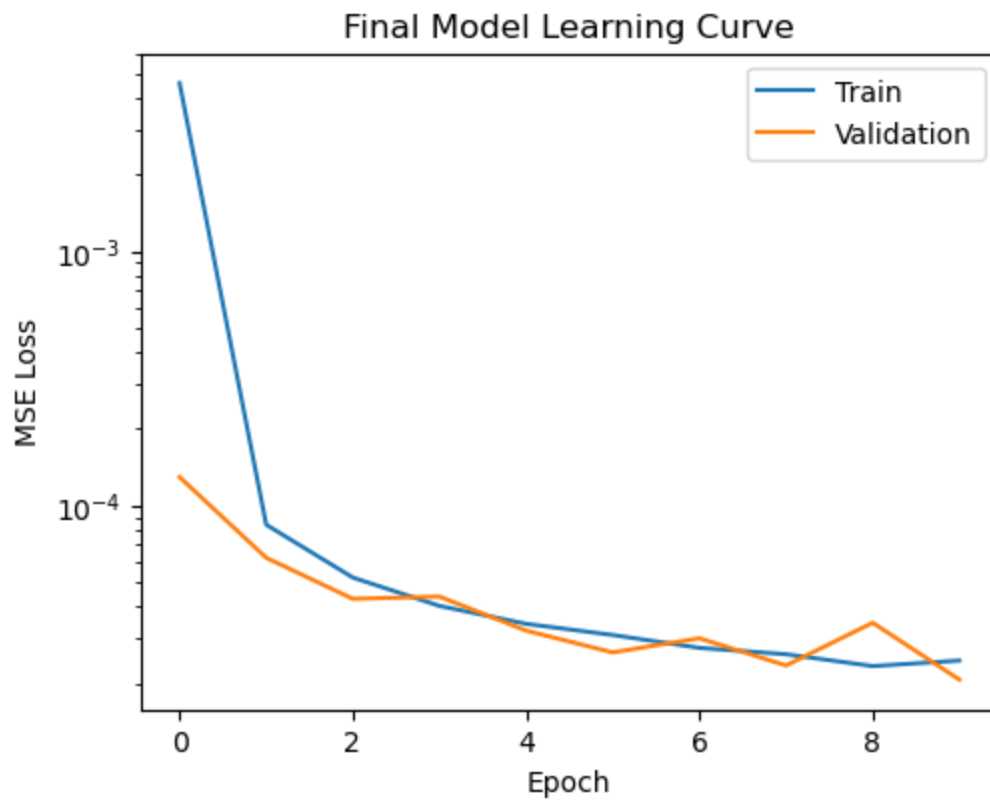
Resulting Validation MAE: 1.7219

--- Testing: Layers=[128, 64], LR=0.001, Batch=4096 ---

Sequential - Number of parameters: 230404

Initialize training data...

100% | ██████████ | 10/10 [00:50<00:00, 5.01s/it]



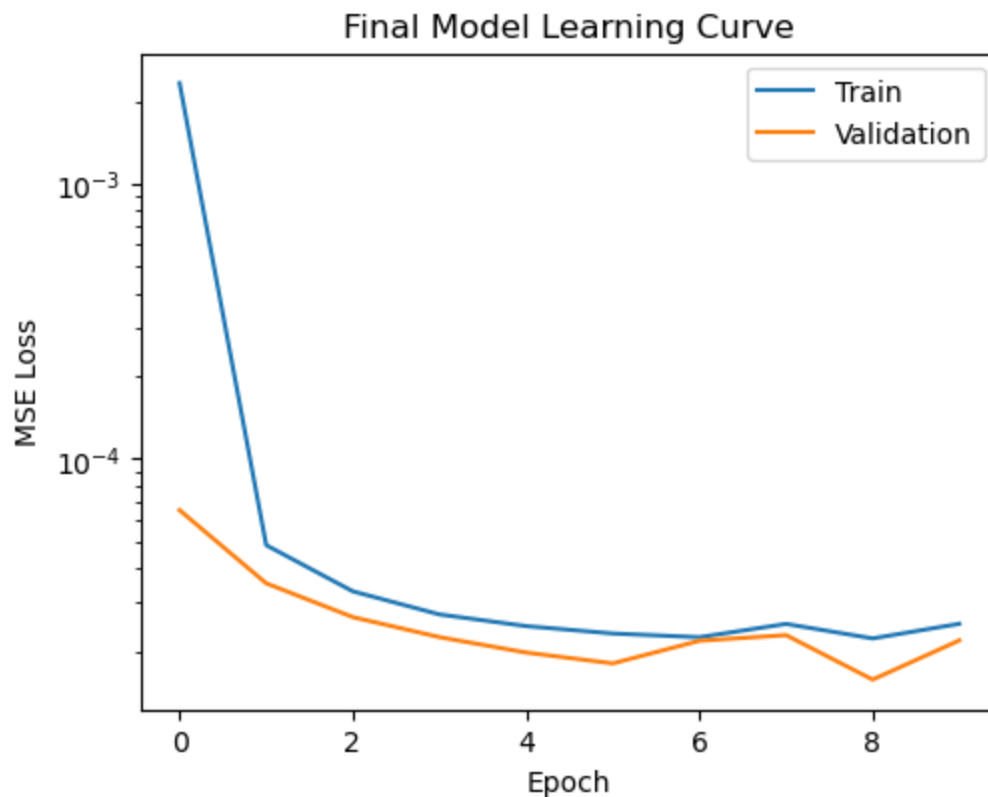
Resulting Validation MAE: 1.8141

--- Testing: Layers=[128, 64], LR=0.001, Batch=2048 ---

Sequential - Number of parameters: 230404

Initialize training data...

100% |██████████| 10/10 [00:57<00:00, 5.78s/it]



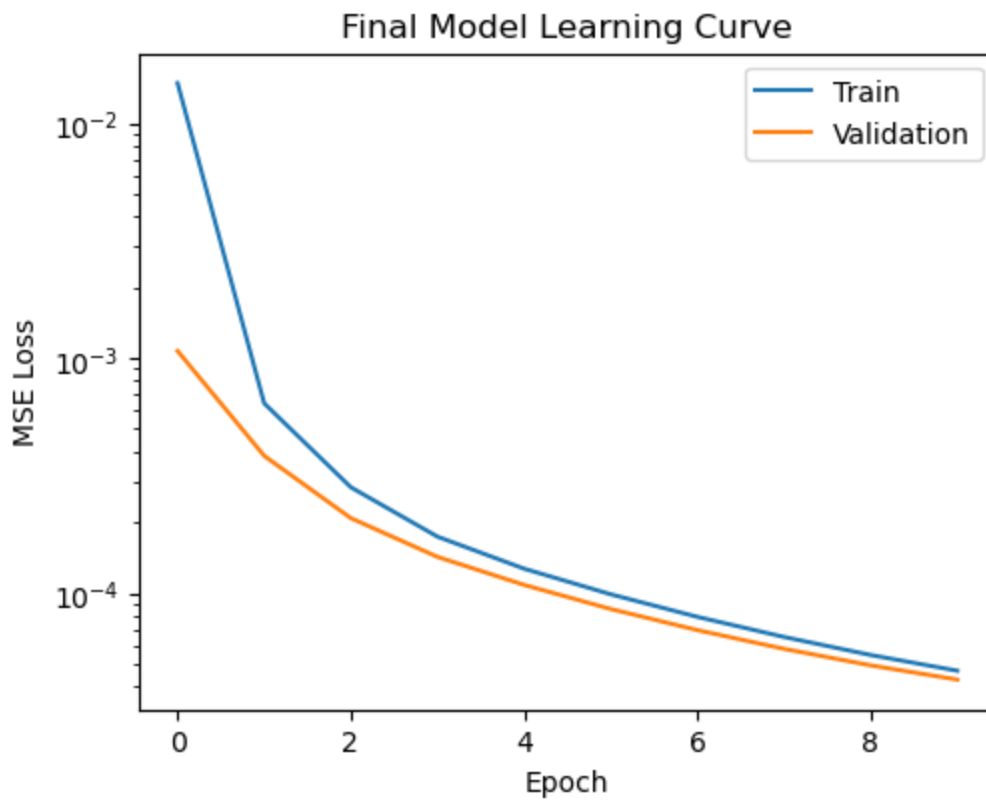
Resulting Validation MAE: 1.6053

--- Testing: Layers=[128, 64], LR=0.0001, Batch=4096 ---

Sequential - Number of parameters: 230404

Initialize training data...

100% |██████████| 10/10 [00:49<00:00, 5.00s/it]



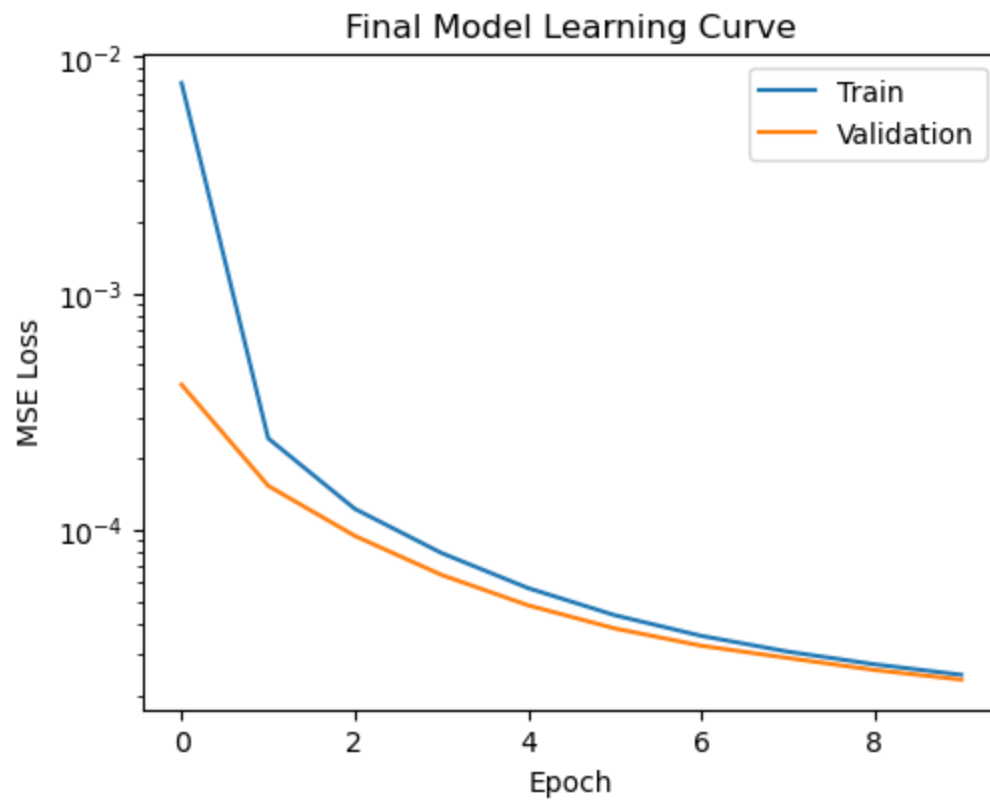
Resulting Validation MAE: 2.2902

--- Testing: Layers=[128, 64], LR=0.0001, Batch=2048 ---

Sequential - Number of parameters: 230404

Initialize training data...

100% |██████████| 10/10 [01:01<00:00, 6.11s/it]



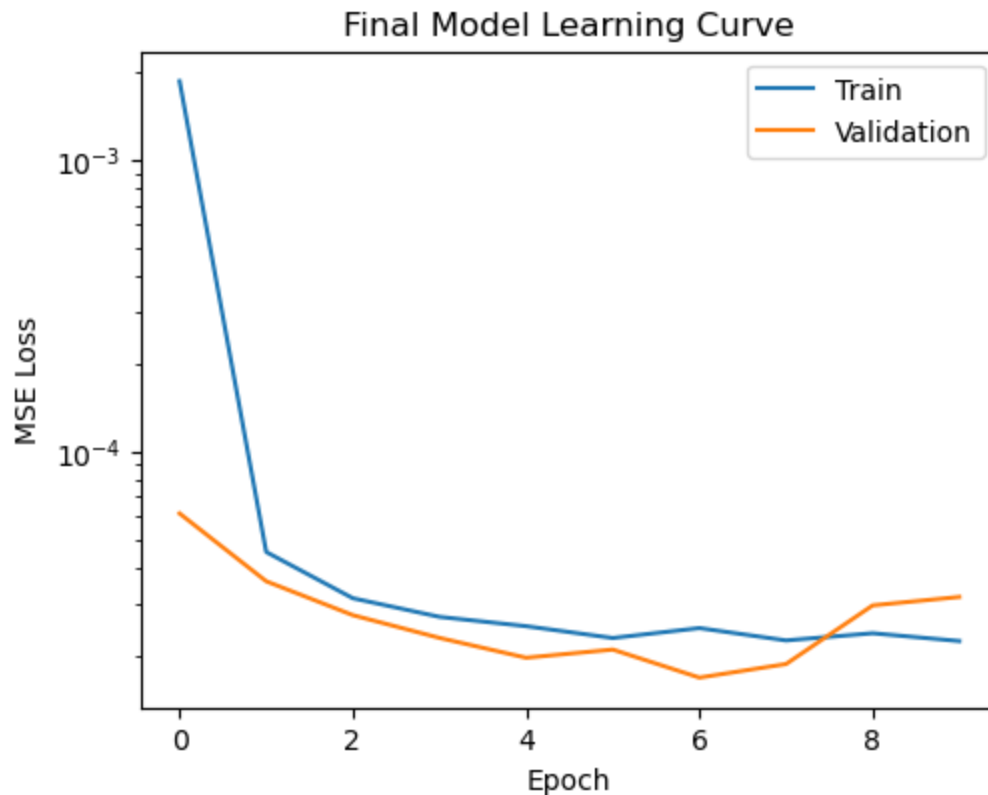
Resulting Validation MAE: 1.8300

--- Testing: Layers=[256, 128], LR=0.001, Batch=4096 ---

Sequential - Number of parameters: 526340

Initialize training data...

100% |██████████| 10/10 [00:51<00:00, 5.16s/it]



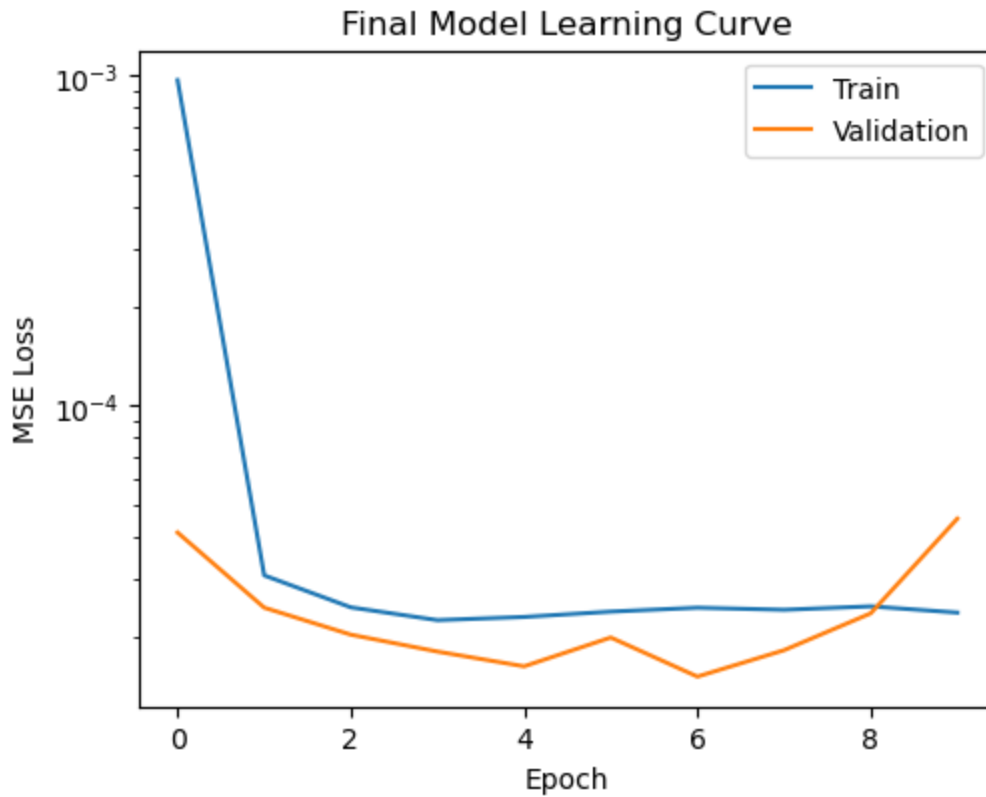
Resulting Validation MAE: 1.6659

--- Testing: Layers=[256, 128], LR=0.001, Batch=2048 ---

Sequential - Number of parameters: 526340

Initialize training data...

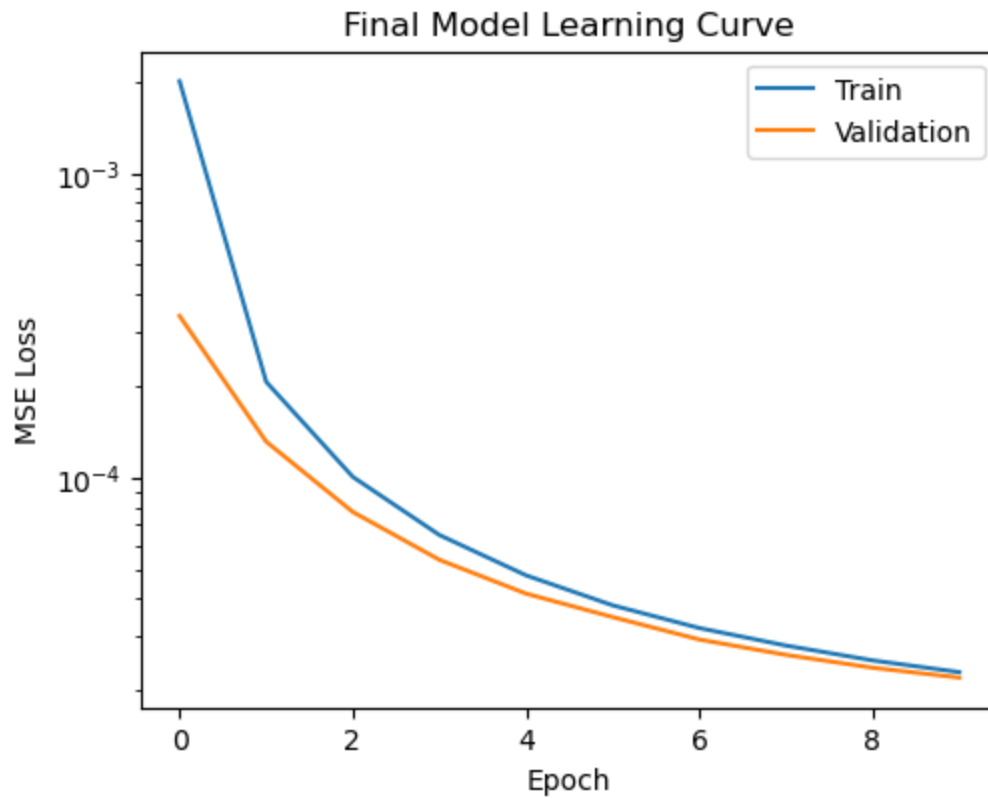
100% |██████████| 10/10 [01:02<00:00, 6.27s/it]



Resulting Validation MAE: 1.6260

--- Testing: Layers=[256, 128], LR=0.0001, Batch=4096 ---
Sequential - Number of parameters: 526340
Initialize training data...

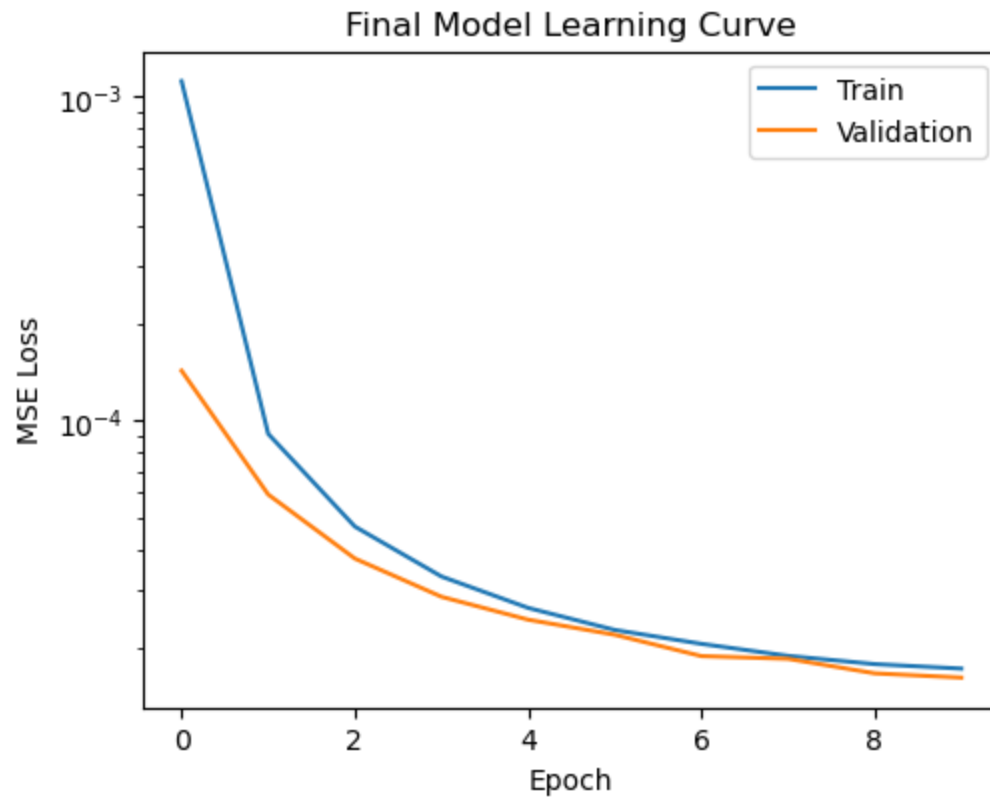
100%|██████████| 10/10 [00:51<00:00, 5.10s/it]



Resulting Validation MAE: 1.7395

--- Testing: Layers=[256, 128], LR=0.0001, Batch=2048 ---
Sequential - Number of parameters: 526340
Initialize training data...

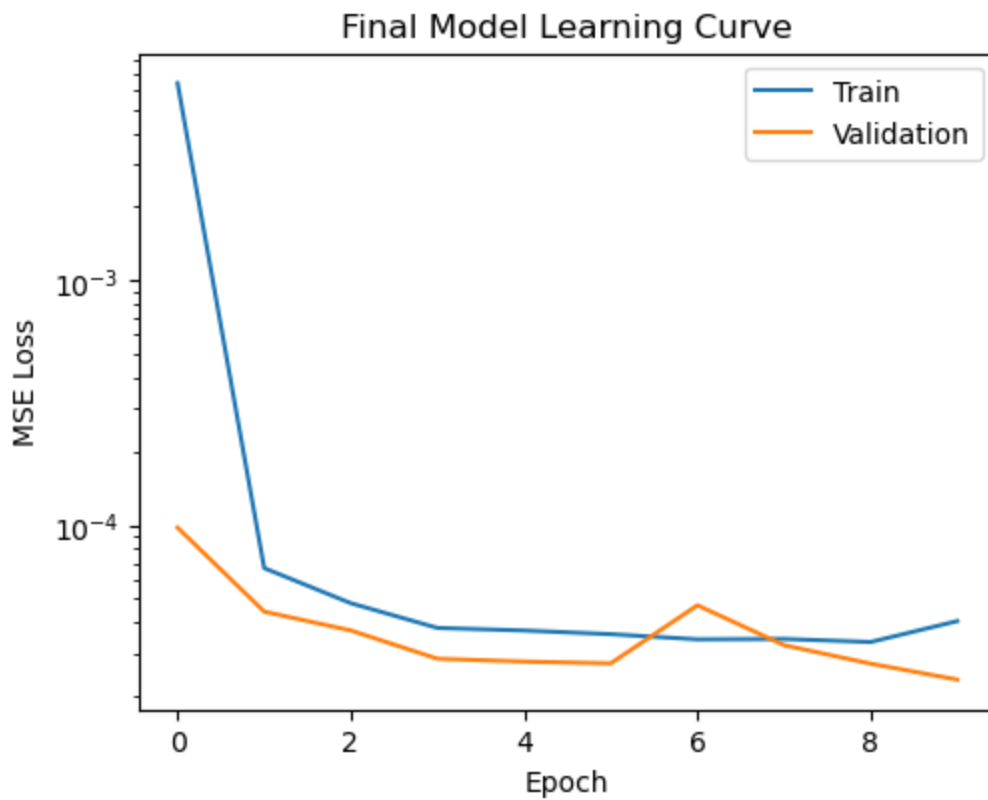
100% |██████████| 10/10 [01:01<00:00, 6.18s/it]



Resulting Validation MAE: 1.5669

--- Testing: Layers=[256, 192, 128, 64], LR=0.001, Batch=4096 ---
Sequential - Number of parameters: 723716
Initialize training data...

100% |██████████| 10/10 [00:58<00:00, 5.81s/it]



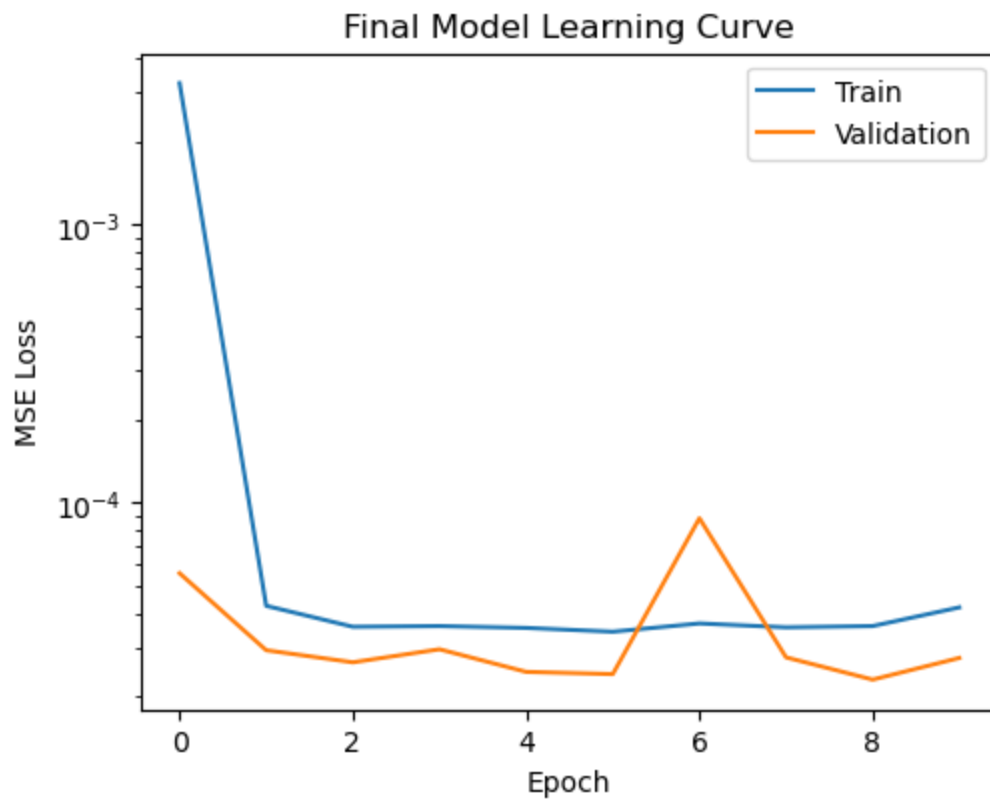
Resulting Validation MAE: 1.9790

--- Testing: Layers=[256, 192, 128, 64], LR=0.001, Batch=2048 ---

Sequential - Number of parameters: 723716

Initialize training data...

100% |██████████| 10/10 [01:12<00:00, 7.21s/it]



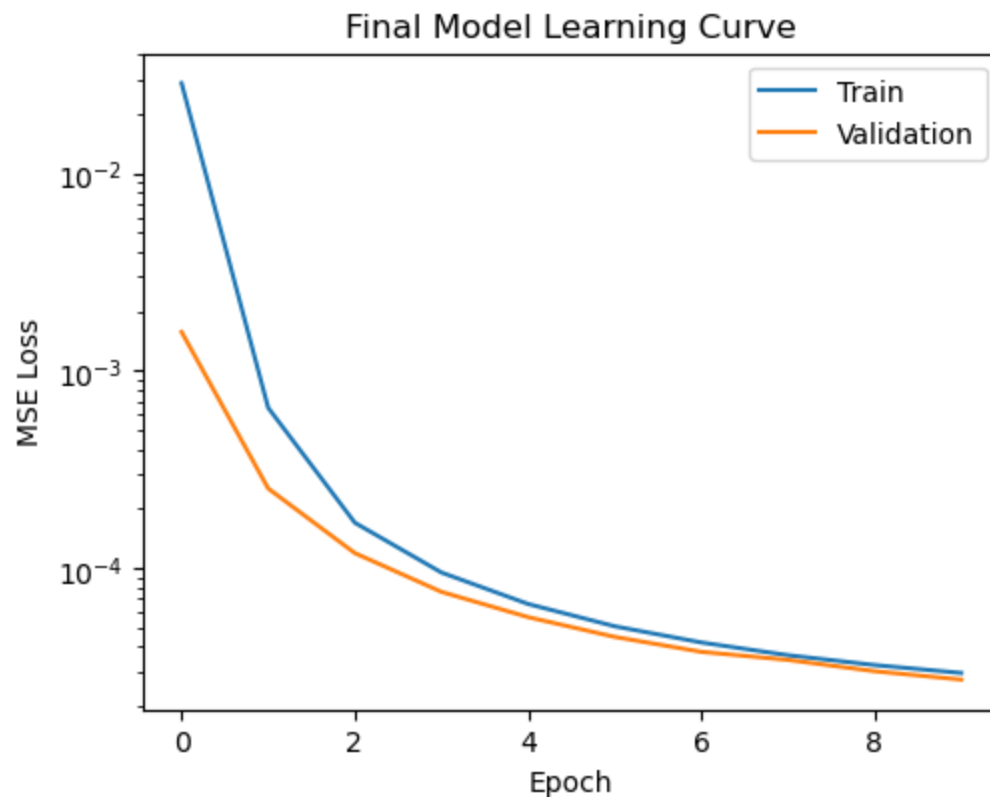
Resulting Validation MAE: 1.9871

--- Testing: Layers=[256, 192, 128, 64], LR=0.0001, Batch=4096 ---

Sequential - Number of parameters: 723716

Initialize training data...

100%|██████████| 10/10 [01:03<00:00, 6.37s/it]



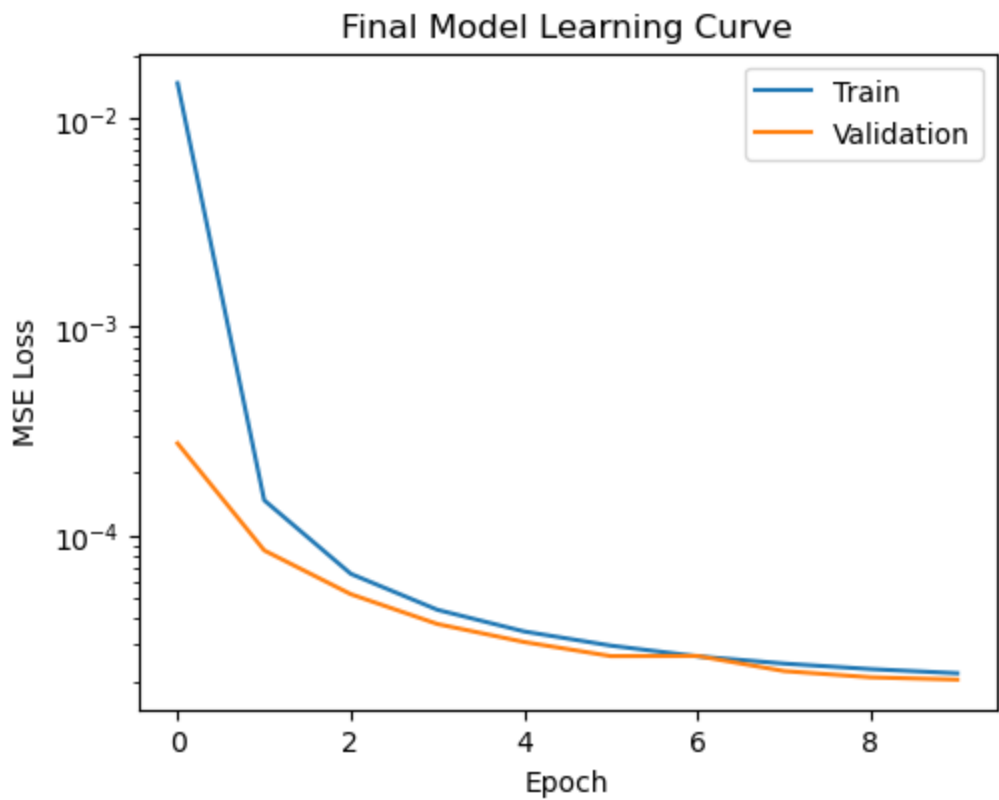
Resulting Validation MAE: 2.0005

--- Testing: Layers=[256, 192, 128, 64], LR=0.0001, Batch=2048 ---

Sequential - Number of parameters: 723716

Initialize training data...

100%|██████████| 10/10 [01:08<00:00, 6.84s/it]



Resulting Validation MAE: 1.8503

```
In [75]: print(f"{'Layers':<25} | {'LR':<7} | {'Batch':<6} | {'RMSE'}")
print("-" * 55)
for r in tuning_results:
    print(f"{r['hidden_layers']:<25} | {r['learning_rate']:<7} | {r['batch_s
```

Layers	LR	Batch	RMSE
[128]	0.001	4096	3.1593
[128]	0.001	2048	2.6679
[128]	0.0001	4096	3.4976
[128]	0.0001	2048	2.7268
[128, 64]	0.001	4096	2.8507
[128, 64]	0.001	2048	2.4922
[128, 64]	0.0001	4096	4.1091
[128, 64]	0.0001	2048	3.0303
[256, 128]	0.001	4096	2.5707
[256, 128]	0.001	2048	2.4451
[256, 128]	0.0001	4096	2.9458
[256, 128]	0.0001	2048	2.5213
[256, 192, 128, 64]	0.001	4096	3.0350
[256, 192, 128, 64]	0.001	2048	3.0156
[256, 192, 128, 64]	0.0001	4096	3.2775
[256, 192, 128, 64]	0.0001	2048	2.8359

```
In [91]: #KEY
# H = 0
# C = 1
# N = 2
# O = 3
```

```

def evaluate_by_heavy_atoms(trainer, data):
    data_loader = data.collate(trainer.batch_size).cache()
    hartree2kcalmol = 627.5094738898777

    errors_by_heavy_atoms = {}

    trainer.model.eval()

    with torch.no_grad():
        for batch in data_loader:
            species = batch["species"].to(device)
            coords = batch["coordinates"].to(device)
            true_energies = batch["energies"].to(device).float()

            _, pred_energies = trainer.model((species, coords))

            errors = torch.abs(true_energies - pred_energies).detach().cpu()
            species_cpu = species.detach().cpu().numpy()

            heavy_atom_counts = (species_cpu != 0).sum(axis=1)

            for count, error in zip(heavy_atom_counts, errors):
                count = int(count)
                if count not in errors_by_heavy_atoms:
                    errors_by_heavy_atoms[count] = []
                errors_by_heavy_atoms[count].append(float(error))

    print("MAE by heavy atom count:")
    heavy_atom_summary = {}

    for count in sorted(errors_by_heavy_atoms.keys()):
        mae = np.mean(errors_by_heavy_atoms[count]) * hartree2kcalmol
        rmse = np.sqrt((np.mean(errors_by_heavy_atoms[count]** 2)) * hartree2kcalmol)
        n = len(errors_by_heavy_atoms[count])
        heavy_atom_summary[count] = {
            "n_samples": n,
            "mae_kcalmol": mae,
            "rmse_kcalmol": rmse
        }
        print(f"{count} heavy atoms: MAE = {mae:.4f}, RMSE = {rmse:.4f}, n = {n}")

    return heavy_atom_summary

heavy_atom_summary = evaluate_by_heavy_atoms(final_trainer, test_data)

```

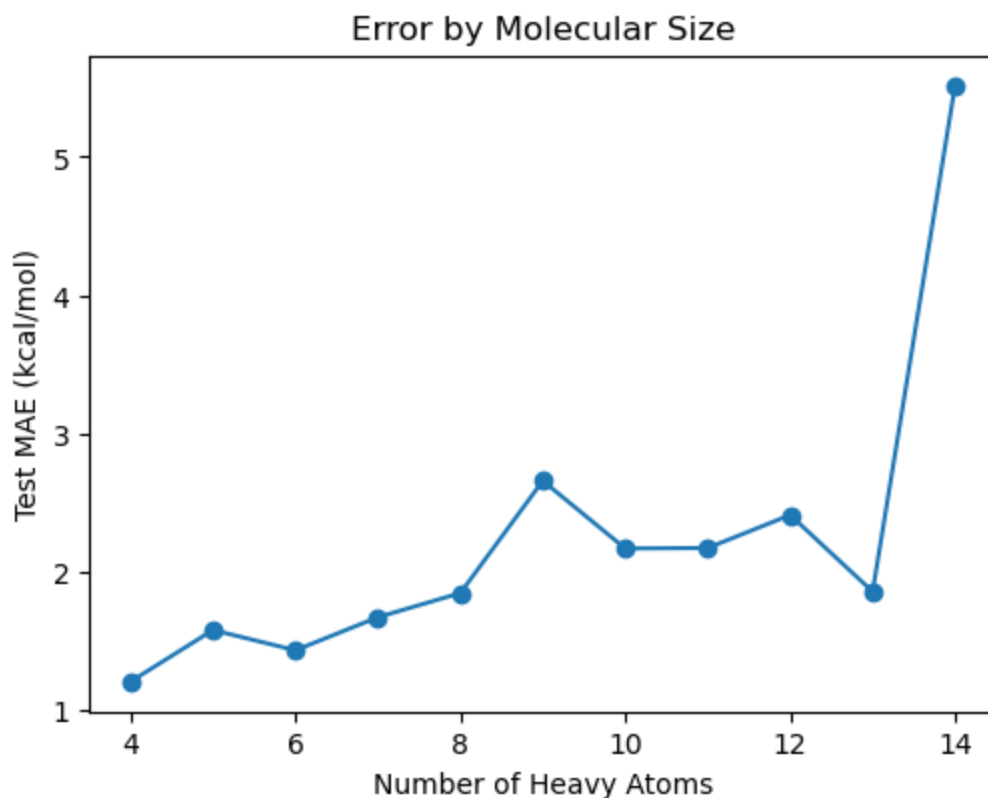
MAE by heavy atom count:

4 heavy atoms: MAE = 1.2011, RMSE = 1.2011, n = 1693
5 heavy atoms: MAE = 1.5746, RMSE = 1.5746, n = 6368
6 heavy atoms: MAE = 1.4278, RMSE = 1.4278, n = 12823
7 heavy atoms: MAE = 1.6672, RMSE = 1.6672, n = 10010
8 heavy atoms: MAE = 1.8423, RMSE = 1.8423, n = 20898
9 heavy atoms: MAE = 2.6552, RMSE = 2.6552, n = 9274
10 heavy atoms: MAE = 2.1650, RMSE = 2.1650, n = 14244
11 heavy atoms: MAE = 2.1686, RMSE = 2.1686, n = 4801
12 heavy atoms: MAE = 2.4075, RMSE = 2.4075, n = 4505
13 heavy atoms: MAE = 1.8527, RMSE = 1.8527, n = 1350
14 heavy atoms: MAE = 5.5200, RMSE = 5.5200, n = 523

MAE heavy atom plot

```
In [93]: counts = sorted(heavy_atom_summary.keys())
maes = [heavy_atom_summary[c]["mae_kcalmol"] for c in counts]

fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
ax.plot(counts, maes, marker="o")
ax.set_xlabel("Number of Heavy Atoms")
ax.set_ylabel("Test MAE (kcal/mol)")
ax.set_title("Error by Molecular Size")
plt.show()
```



Testing Layers=[256, 128], LR=0.0001, Batch=2048

```
In [77]: FINAL_CONFIG2 = {
    "hidden_layers": [256, 128],
    "activation": "relu",
    "batch_size": 2048,
    "learning_rate": 1e-4,
    "epoch": 50,
    "l2": 1e-5,
    "use_batchnorm": True,
    "use_huber": True,
    "grad_clip": 1.0,
    "use_cosine_lr": True,
}

print("Improved final configuration:")
for k, v in FINAL_CONFIG2.items():
    print(f"  {k}: {v}")
```

```
Improved final configuration:
hidden_layers: [256, 128]
activation: relu
batch_size: 2048
learning_rate: 0.0001
epoch: 50
l2: 1e-05
use_batchnorm: True
use_huber: True
grad_clip: 1.0
use_cosine_lr: True
```

```
In [78]: set_seed(42)

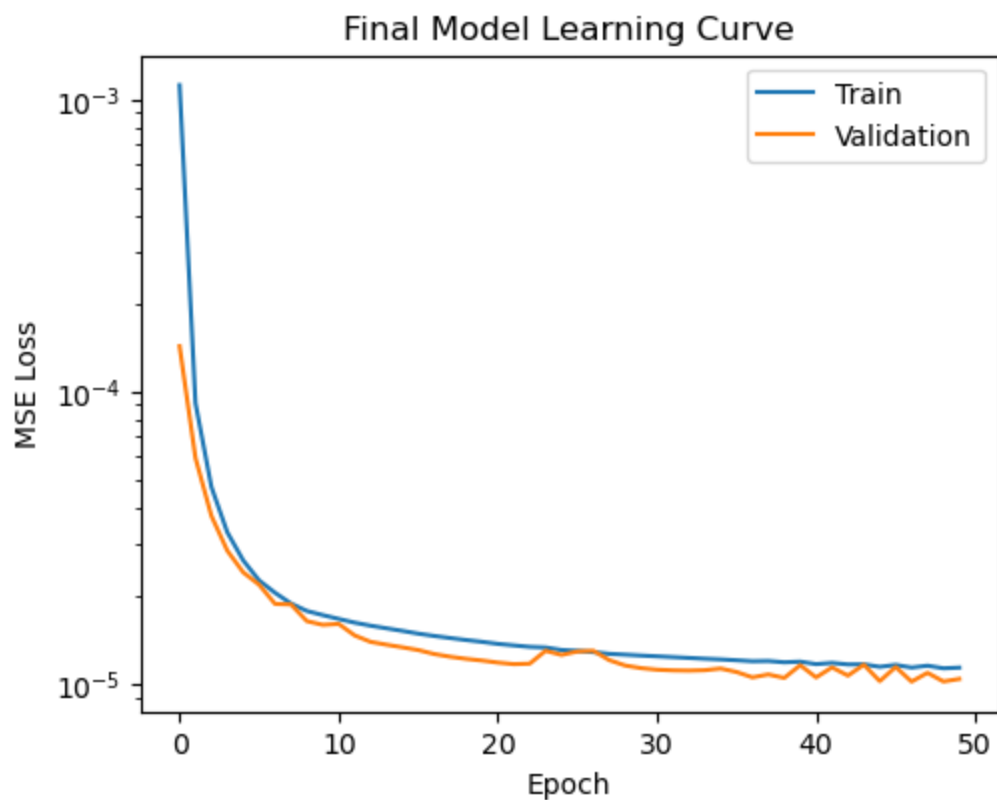
final_model2 = build_model(
    hidden_layers=FINAL_CONFIG2["hidden_layers"],
    activation=FINAL_CONFIG2["activation"]
)

final_trainer2 = ANITrainer(
    model=final_model2,
    batch_size=FINAL_CONFIG2["batch_size"],
    learning_rate=FINAL_CONFIG2["learning_rate"],
    epoch=FINAL_CONFIG2["epoch"],
    l2=FINAL_CONFIG2["l2"]
)

final_train_losses, final_val_losses = final_trainer2.train(
    train_data=train_data,
    val_data=val_data,
    early_stop=True,
    draw_curve=True
)
```

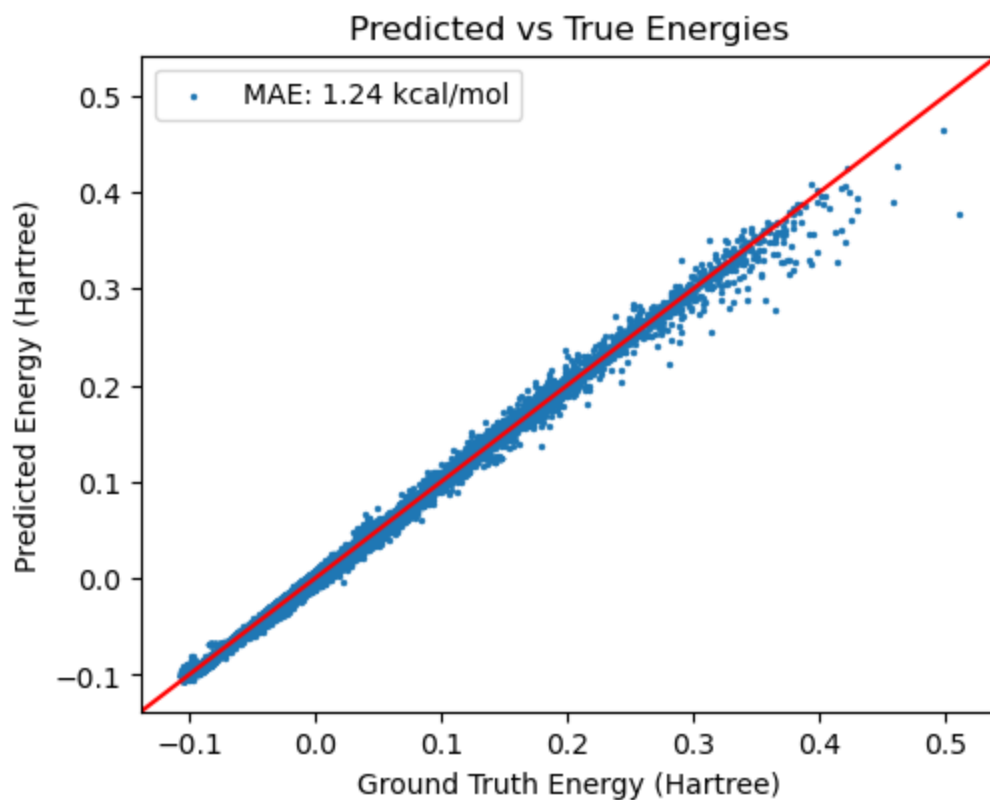
```
Sequential - Number of parameters: 526340
Initialize training data...
```

```
100%|██████████| 50/50 [05:03<00:00, 6.07s/it]
```



```
In [79]: final_test_mse2 = final_trainer2.evaluate(test_data, draw_plot=True)
final_test_mae2 = final_trainer2.evaluate_mae_kcalmol(test_data)
final_test_rmse2 = final_trainer2.evaluate_rmse_kcalmol(test_data)

print("\nFinal model evaluation on held-out test set:")
print(f"Final Test MSE2: {final_test_mse2:.8f}")
print(f"Final Test MAE2: {final_test_mae2:.4f} kcal/mol")
print(f"  Final Test RMSE2: {final_test_rmse2:.4f} kcal/mol")
```



MAE: 1.2397 kcal/mol

Final model evaluation on held-out test set:

Final Test MSE2: 0.00000940

Final Test MAE2: 1.2397 kcal/mol

Final Test RMSE2: 1.9238 kcal/mol

```
In [81]: conclusions2 = {
    "hidden_layers": FINAL_CONFIG2["hidden_layers"],
    "activation": FINAL_CONFIG2["activation"],
    "batch_size": FINAL_CONFIG2["batch_size"],
    "learning_rate": FINAL_CONFIG2["learning_rate"],
    "epochs": FINAL_CONFIG2["epoch"],
    "l2_regularization": FINAL_CONFIG2["l2"],
    "final_test_mse": final_test_mse2,
    "final_test_mae_kcalmol": final_test_mae2
}
print("\nFinal configuration conclusions2/summary:")
for k, v in conclusions2.items():
    print(f"{k}: {v}")
```

Final configuration conclusions2/summary:

hidden_layers: [256, 128]

activation: relu

batch_size: 2048

learning_rate: 0.0001

epochs: 50

l2_regularization: 1e-05

final_test_mse: 9.398918876296946e-06

final_test_mae_kcalmol: 1.239695971364999

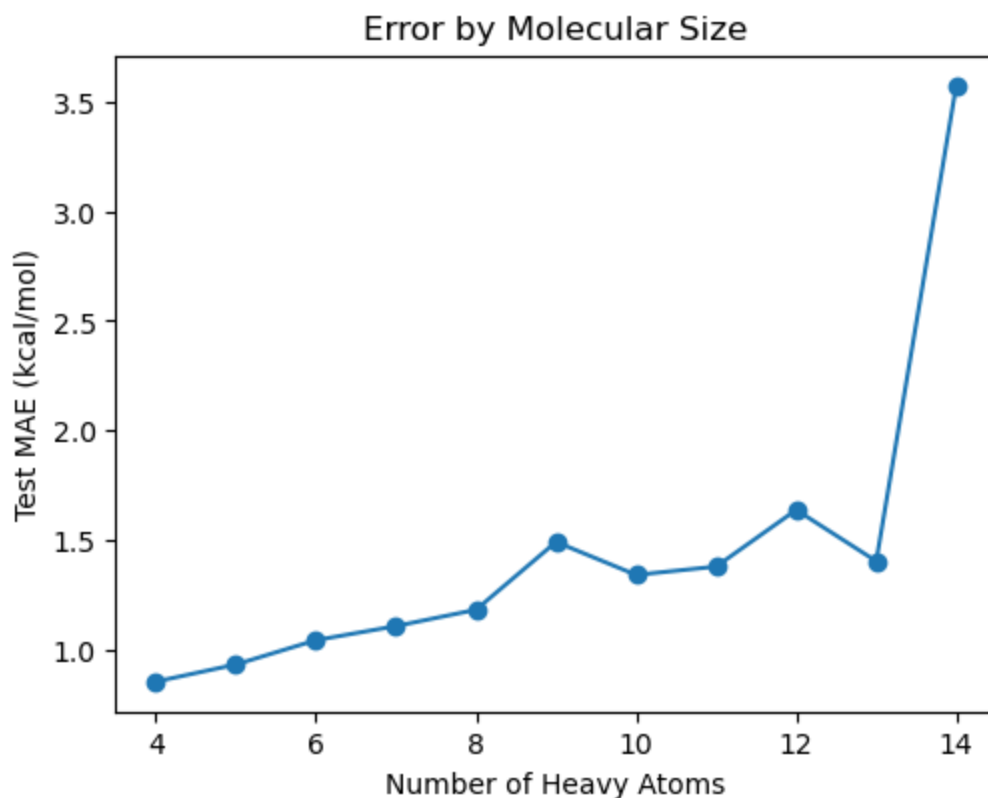
Heavy Atom MAE with Final 2 model

```
In [94]: heavy_atom_summary = evaluate_by_heavy_atoms(final_trainer2, test_data)
counts = sorted(heavy_atom_summary.keys())
maes = [heavy_atom_summary[c]["mae_kcalmol"] for c in counts]

fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
ax.plot(counts, maes, marker="o")
ax.set_xlabel("Number of Heavy Atoms")
ax.set_ylabel("Test MAE (kcal/mol)")
ax.set_title("Error by Molecular Size")
plt.show()
```

MAE by heavy atom count:

4 heavy atoms: MAE = 0.8539, RMSE = 0.8539, n = 1693
5 heavy atoms: MAE = 0.9320, RMSE = 0.9320, n = 6368
6 heavy atoms: MAE = 1.0430, RMSE = 1.0430, n = 12823
7 heavy atoms: MAE = 1.1072, RMSE = 1.1072, n = 10010
8 heavy atoms: MAE = 1.1818, RMSE = 1.1818, n = 20898
9 heavy atoms: MAE = 1.4911, RMSE = 1.4911, n = 9274
10 heavy atoms: MAE = 1.3407, RMSE = 1.3407, n = 14244
11 heavy atoms: MAE = 1.3787, RMSE = 1.3787, n = 4801
12 heavy atoms: MAE = 1.6374, RMSE = 1.6374, n = 4505
13 heavy atoms: MAE = 1.4032, RMSE = 1.4032, n = 1350
14 heavy atoms: MAE = 3.5748, RMSE = 3.5748, n = 523



RMSE Heavy Atom on Final 2 model

```
In [98]: heavy_atom_summary = evaluate_by_heavy_atoms(final_trainer2, test_data)
```

MAE by heavy atom count:

```
4 heavy atoms: MAE = 0.8539, RMSE = 0.8539, n = 1693
5 heavy atoms: MAE = 0.9320, RMSE = 0.9320, n = 6368
6 heavy atoms: MAE = 1.0430, RMSE = 1.0430, n = 12823
7 heavy atoms: MAE = 1.1072, RMSE = 1.1072, n = 10010
8 heavy atoms: MAE = 1.1818, RMSE = 1.1818, n = 20898
9 heavy atoms: MAE = 1.4911, RMSE = 1.4911, n = 9274
10 heavy atoms: MAE = 1.3407, RMSE = 1.3407, n = 14244
11 heavy atoms: MAE = 1.3787, RMSE = 1.3787, n = 4801
12 heavy atoms: MAE = 1.6374, RMSE = 1.6374, n = 4505
13 heavy atoms: MAE = 1.4032, RMSE = 1.4032, n = 1350
14 heavy atoms: MAE = 3.5748, RMSE = 3.5748, n = 523
```

Comparing ANI-1 vs. Model

```
In [99]: paper_train_rmse = 1.2
paper_val_rmse = 1.3
paper_test_rmse = 1.3
paper_transfer_rmse = 1.8

print("\nComparison to ANI-1 paper benchmarks:")
print(f"ANI-1 paper train RMSE: {paper_train_rmse} kcal/mol")
print(f"ANI-1 paper validation RMSE: {paper_val_rmse} kcal/mol")
print(f"ANI-1 paper test RMSE: {paper_test_rmse} kcal/mol")
print(f"ANI-1 transferability benchmark RMSE: {paper_transfer_rmse} kcal/mol")
```

Comparison to ANI-1 paper benchmarks:

ANI-1 paper train RMSE: 1.2 kcal/mol

ANI-1 paper validation RMSE: 1.3 kcal/mol

ANI-1 paper test RMSE: 1.3 kcal/mol

ANI-1 transferability benchmark RMSE: 1.8 kcal/mol

```
In [102]: print("\nCheckpoint 5 complete.")
print(f"Final Test MAE: {final_test_mae2:.4f} kcal/mol")
print(f"Final Test RMSE: {final_test_rmse2:.4f} kcal/mol")
```

Checkpoint 5 complete.

Final Test MAE: 1.2397 kcal/mol

Final Test RMSE: 1.9238 kcal/mol

```
In [101]: heavyten_results = heavy_atom_summary[10]
heavy_8 = heavy_atom_summary[8]
print (heavyten_results)
print (heavy_8)
```

```
{'n_samples': 14244, 'mae_kcalmol': 1.3407116398260583, 'rsme_kcalmol': 1.3407116398260583}
```

```
{'n_samples': 20898, 'mae_kcalmol': 1.1817655782366672, 'rsme_kcalmol': 1.1817655782366672}
```

This project model is smaller and simpler than the full ANI-1 architecture as the final test MAE lies below 2 kcal/mol. Standing only at 1.2397 kcal/mol, it is lower than the ANI-1

paper benchmark of 1.8 kcal/mol, seeing as we run our model on 50 epochs versus the 100 epochs that the ANI-1 original architecture did. Even as molecules get larger (up to 13 heavy atoms), the MAE stays below the 2 kcal/mol line which we see in the "Error by molecule Size" in our tests compared to Figure 4 page 3196, where 10 heavy atoms had a transferability RSME error of 1.8 kcal/mol. Though out RMSE is higher, for up to 13 heavy atoms, our RMSE still ranks below the 1.8 RMSE marker. The largest variability was with 14 heavy atoms.

In []: